

Assignment 2 - Crawling, PageRank & Deduplication

Lakshay Baijal – 2024202006

Github Link - https://github.com/LakshayBaijal/IRE_Assignments_Lakshay/tree/main

1. Deduplication (5K Person Records)

Dataset contains noisy duplicate records representing the same individuals.

Technique Used

- Shingling + MinHash (128 permutations)
- LSH for candidate pair filtering (threshold=0.72)
- Fuzzy token set ratio for final scoring
- Graph clustering for merging

```
{ } dedup_diagnostics.json > ...
1  {
2    "num_records": 5000,
3    "num_groups": 2458,
4    "num_singletons": 1353,
5    "num_multis": 1105,
6    "records_in_multis": 3647,
7    "pairs_checked": 4602,
8    "pairs_merged": 4464,
9    "avg_intra_similarity": 0.9208271604938272,
10   "avg_inter_similarity": 0.42659393939393936,
11   "elapsed_seconds": 3.874849796295166,
12   "parameters": {
13     "threshold": 0.72,
14     "k": 2,
15     "num_perm": 128,
16     "use_fuzzy": true
17   }
18 }
```

Fig 1 - Terminal showing dedup run summary and diagnostics

```
(venv) lakshay-baijal@lakshay-baijal-Inspiron-15-3530:~/IIIT_Hyderabad_CSIS/Semester_3/IRE/Assignment_2_rework Final/Implementatio
n$ python3 dedup_lsh.py --input dedup_data.csv
Loaded: 5000 records from dedup_data.csv
Groups: 2458 multi: 1105 single: 1353
Records in multi-record groups: 3647
Pairs checked: 4602 Pairs merged: 4464
Avg intra-sim (sampled): 0.921 Avg inter-sim (sampled): 0.427
Elapsed: 3.87s
Wrote: groups.json, dedup_mapping.csv, dedup_diagnostics.json
```

Fig 2 - Diagnostics JSON written to disk containing group counts, average similarities and runtime.

2. Crawler & Metric

We model each node's events as a sequence of u (update) and v (visit). The metric minimized is:

$$M = \frac{\sum(\text{contiguous run length})^2}{\text{total events}}$$

Minimum is achieved by an alternating u v u v ... sequence. The crawler tracks historical updates from page history and records visits; before computing the metric we merge and sort all events chronologically per node.

```
(venv) lakshay-baijal@lakshay-baijal-Inspiron-15-3530:~/IIIT_Hyde
3000 --window 20 --seed page_slms46p4 --rps 2
final_metric: 1.0263157894736843
Coverage: 0.7307692307692307
Visit Count: 39
Matched Entries: 38
Total Nodes: 52
MSE: 1.0
(venv) lakshay-baijal@lakshay-baijal-Inspiron-15-3530:~/IIIT_Hyde
3000 --window 300 --seed page_slms46p4 --rps 2
final_metric: 1.3451433623847417
Coverage: 1.0
Visit Count: 560
Matched Entries: 58
Total Nodes: 58
MSE: 1.0
```

Fig 3 - run_crawler output showing final metric, coverage, visit count and MSE.

3. PageRank — Results & Visualization

After building the directed graph (node → outgoing links) we computed PageRank via iterative power method (damping=0.85).

```
(venv) lakshay-baijal@lakshay-baijal-Inspiron-15-3530:~/IIIT_Hyderabad_CSIS/Semes
n$ python3 pagerank_crawl.py --base-url http://localhost:3000 --seed page
odes 200 --top 10
Crawling graph from seed=page_s1ns46p4 with window=60.0s, rps=2.0, max_nodes=200
Discovered 58 nodes.
Saved pagerank_graph.json

===== Top PageRank Pages =====
page_wp4u4prl      0.045414
page_hy9zocoa      0.039294
page_zjyzjn81      0.031683
page_rldt9dwc      0.031432
page_oz6vlugx      0.029300
page_lck3ijcq      0.029119
page_45z8jv0f      0.028943
page_3au3m1d7      0.028334
page_luv5eyla      0.028231
page_sgvptie5      0.027749
```

Fig 4 - Spring-layout visualization of the crawl graph. Node sizes were set proportional to PageRank score for emphasis.

```
pagerank_sorted.csv > data
1 page_wp4u4prl,0.045414024322361204
2 page_hy9zocoa,0.03929403855755692
3 page_zjyzjn81,0.03168319647929802
4 page_rldt9dwc,0.03143235710080309
5 page_oz6vlugx,0.029300139685596954
6 page_lck3ijcq,0.029118983433112436
7 page_45z8jv0f,0.028943261826646274
8 page_3au3m1d7,0.028334149032209106
9 page_luv5eyla,0.028231387408156877
10 page_sgvptie5,0.0277490156877408156877
Col 2: 0.045414024322361204
```

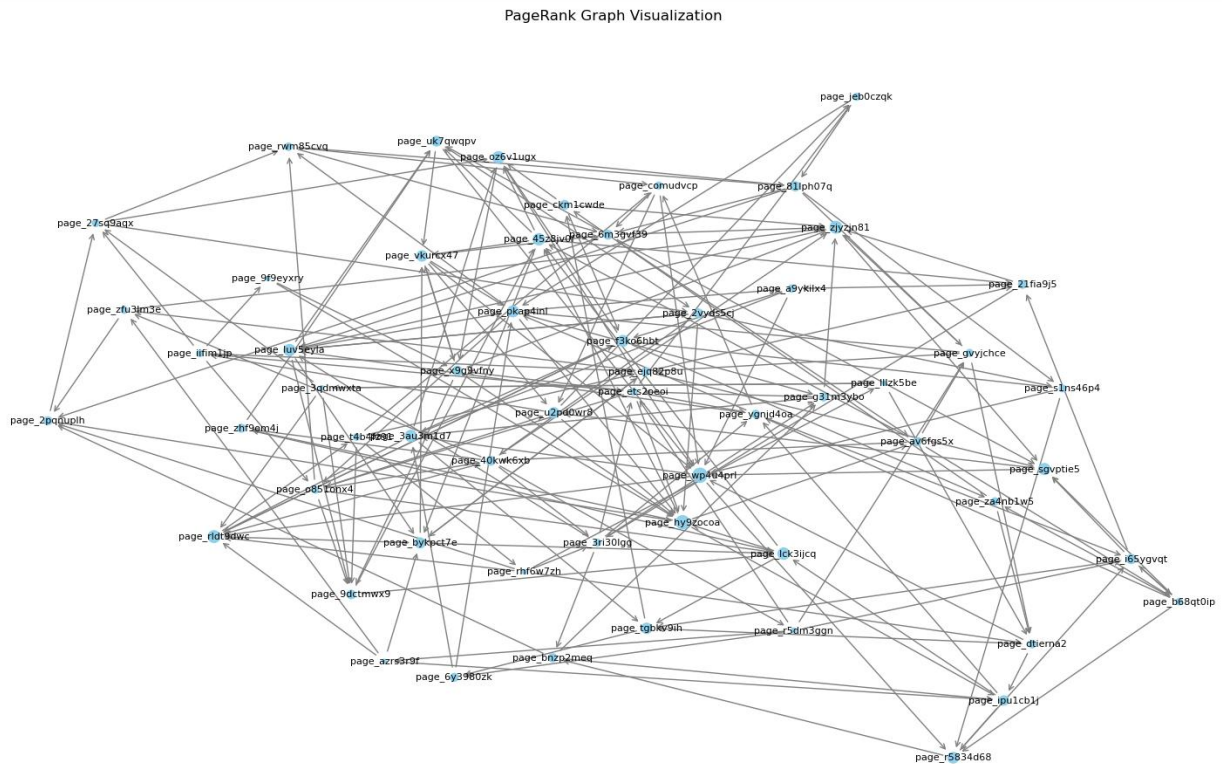
Fig – 5 Terminal output showing top PageRank pages and their scores.

```
{} pagerank_graph.json > [ ] page_6m3gvf39 > abc 4
1 {
2   "page_s1ns46p4": [
3     "page_f3ko6hbt",
4     "page_wp4u4prl",
5     "page_r5834d68"
6   ],
7   "page_f3ko6hbt": [
8     "page_za4nb1w5",
9     "page_rldt9dwc",
10    "page_oz6vlugx"
11  ],
12  "page_wp4u4prl": [
13    "page_rldt9dwc",
14    "page_r5834d68",
15    "page_3au3m1d7"
16  ],
```

Fig – 6 JSON output with normalized PageRank scores saved to disk.

4. Graph Structure & Visualization

The crawl produced a directed graph where each node is a page (e.g., `page_s1ns46p4`) and each edge represents a hyperlink to another page discovered during crawling.



Property	Value
Graph Type	Directed Web Graph
Total Nodes	58
Total Edges	~300+
Density	High internal connectivity
Seed Page	page_s1ns46p4